ИТМО

# Разработка инструмента фаззинга сервисов на userver

Автор: Скрябин Иван Александрович

Научный руководитель: Степанов Даниил Сергеевич

# Актуальность фаззинга

ИТМО

## Поиск ошибок до выпуска сервиса

Фаззинг автоматически генерирует и изменяет входные данные, чтобы находить ошибки за пределами заранее написанных тестов.

## Фаззинг в промышленной разработке

OSS-Fuzz: 13 000+ уязвимостей в 1 000 проектах к маю 2025 [1]. Microsoft применяет OneFuzz в Windows и Edge [2].

## Высокий порог подключения к сервису

Подключение фаззинга требует изоляции окружения и контроля состояния между тестовыми примерами.

Источники: [1] Google OSS-Fuzz  [2] Microsoft OneFuzz

# userver и testsuite

## userver: разработка сервисов

Открытый асинхронный C++-фреймворк для микросервисов.
Поддерживает HTTP, gRPC, работу с БД и брокерами.
Многие сервисы на userver работают в продакшене [3].

## testsuite: тестирование сервисов

Python-фреймворк на основе pytest для функциональных тестов.
Запускает сервис, готовит данные и подменяет внешние API.

Задача: находить ошибки в userver-сервисах до выпуска,
используя testsuite для управления тестовым окружением.

Источники: userver  testsuite  [3] userver в production

# Обзор подходов и аналогов

| Инструмент | Основной подход | Вопрос для нашего проекта |
|---|---|---|
| **Schemathesis** | OpenAPI, Python,<br>stateful-цепочки | Async testsuite<br>Покрытие C++ |
| **RESTler** | Зависимости между<br>запросами и цепочки | Изоляция окружения<br>Воспроизведение |
| **EvoMaster** | Генерация API-тестов,<br>black-box режим | Поиск ошибок<br>на C++-стендах |
| **WuppieFuzz** | Фаззинг REST<br>с обратной связью | Покрытие C++<br>Сброс состояния |

**Фокус работы: совместить поиск цепочек и данных с изоляцией testsuite и обратной связью из C++.**

# Цель и гипотеза

## Цель работы

Разработать и оценить инструмент grey-box фаззинга сервисов на userver с интеграцией в testsuite.

## Проверяемая гипотеза

Обратная связь по покрытию C++ и пользовательские сценарии повышают эффективность поиска ошибок.

Проверка: сравнить режимы с этими возможностями и без них при одинаковом бюджете времени.

# Предлагаемый инструмент

ИТМО

### 01 Отдельные запросы

Генерация по API-схеме и проверка контракта ответа.

### 02 Цепочки без пользовательских сценариев

Автоматический выбор порядка, повторов и связей данных.

### 03 Цепочки по сценариям на Python

Use case задаёт стартовую последовательность для мутаций.

Первый прототип: REST / OpenAPI.
Потенциальное расширение: gRPC / Protocol Buffers.

# Пример исследования сценария

**Сценарий разработчика**

```
ticket = prepare(value)
commit(ticket)
```

**Один из вариантов фаззера**

```
ticket = prepare(value)
commit(ticket)
commit(ticket)
```

**Число повторений и входные значения фаззер подбирает самостоятельно.**

Контекст сохраняет связи данных между вызовами.
Поиск включает неожиданный порядок и устаревшие значения.

# Окружение и изоляция

ИТМО

### Окружение как часть входа фаззера

Фаззер варьирует ответы моков, файлы, данные БД
и сообщения брокера по заданным схемам и генераторам.

### Управление через testsuite

Фаззер выбирает действия и данные, testsuite их применяет.
Изменения окружения можно чередовать с вызовами сервиса.

### Изоляция и воспроизведение

Testsuite восстанавливает сервис и зависимости между примерами.
Для повтора сохраняются запросы и изменения окружения.

Все три режима. Отказы зависимостей остаются за рамками текущего этапа.

# Задачи работы

1. Сравнить готовые фаззеры на контрольных стендах.
2. Реализовать генерацию запросов и цепочек со связями данных.
3. Добавить Python API для сценариев и пользовательских проверок.
4. Интегрировать подготовку и сброс окружения с testsuite.
5. Подключить обратную связь по покрытию C++.
6. Реализовать воспроизведение и сокращение находок, затем провести сравнительные эксперименты.

# Текущее состояние проекта

## Уже подготовлены

REST- и gRPC-стенды с пятью классами известных дефектов.
Контракты API, HTTP-моки и функциональные testsuite-тесты.
Контрольные проверки воспроизведения падений.

## Сформулирована концепция

Три режима, данные зависимостей, изоляция и критерии оценки.
Подготовлен список инструментов для сравнения.

**Следующий шаг: первые сравнительные прогоны и выбор основы прототипа.**

# Ожидаемый результат

**Рабочий прототип с Python API, изоляцией окружения и воспроизводимыми примерами найденных ошибок.**

### Минимальная цель на стендах

Падение от одного запроса
Недекларированный HTTP 500
Падение после цепочки команд
Повторный commit с общим ticket
Сочетание запроса и ответа мока

### Критерии сравнения

Доля найденных дефектов
Время до обнаружения
Повторяемость находок
Покрытие и накладные расходы

Сравнение при одинаковых бюджетах и учтённых подсказках

# Спасибо за внимание!

ITMO more than a UNIVERSITY

Ваши контакты